Practical Task 8.1

(Pass Task)

Submission deadline: Monday, May 11
Discussion deadline: Friday, May 29

Instructions

In this task, you need to answer the questions formulated in Parts A-F. You do not need to write any program code.

- **Part A**

Miss Marple is at a train station in a foreign town. She wants to select a hotel that has the *maximum number of shortest paths* from the train station. She thinks that this should reduce the risk of getting lost. Suppose she gives you a city map represented via a graph $G = \langle V, E \rangle$ with $n = |V|$ locations and $m = |E|$ edges connecting the locations. Each edge connecting a pair of directly connected locations has a *unit distance*, say 1. Help Miss Marple to find a proper hotel by designing a $O(n + m)$ runtime algorithm that finds the number of shortest paths between the train station, located at node s and every hotel on the map. You need to convince her that your algorithm is correct, and it does find all possible shortest paths. Your solution must contain an explanation and a pseudocode.

- **Part B**

A communication network, such as the Internet, can be modelled as an undirected graph $G = \langle V, E \rangle$. Here, the vertices of set V are the computers of the network, and the edge set E consists of one edge for each pair of computers that are directly connected. We assume that the edges of G are undirected; that is, if there is a direct connection from computer u to computer v, then there is also a direct connection from computer v to computer u.

It is highly desirable for a communication network graph to be ***connected*** so that every computer on the network can communicate, possibly through a series of relays, with any other computer. But networks can change, with some computers failing and other computers being added to the network. It is useful to have a *testing algorithm* that collects information about the current network graph (vertices and edges) at designated times and determines properties related to connectivity.

Describe, in words and a pseudocode, a ***testing algorithm*** that given an undirected graph $G = \langle V, E \rangle$ representing the current network decides whether the network is connected. A graph (the network) is ***connected*** if there is a path from any node to any other node in the graph. You may assume that G is given in the *adjacency list* format. Your algorithm must run in $O(n + m)$ time, where $n = |V|$ is the number of computers of the network, and $m = |E|$ is the number of connecting edges.

- **Part C**

Given a graph $G = \langle V, E \rangle$, what is to be the runtime complexity of the ***depth-first search*** algorithm, as a function of the number of nodes $n = |V|$ and edges $m = |E|$, if the input graph is represented by an ***adjacency matrix*** instead of an *adjacency list*?

- **Part D**

Consider the directed graph $G = \langle V, E \rangle$ represented by the following cost adjacency matrix:

$$A = \begin{bmatrix} . & 4 & . & . & . & 10 & . & . & . & . \\ . & . & . & 5 & 11 & . & 8 & 10 & . & . \\ . & . & . & . & 9 & . & . & . & . & . \\ 6 & . & 2 & . & . & 2 & . & . & . & . \\ . & . & . & . & 0 & . & . & 0 & . & 3 \\ . & . & . & 3 & . & . & . & . & . & . \\ . & 7 & . & . & . & 2 & . & . & . & . \\ . & . & 10 & . & . & . & . & . & . & . \\ . & . & . & . & . & . & . & 3 & . & . \\ . & . & . & 4 & . & . & . & . & . & . \end{bmatrix}$$

Assume that element a_{ij} positioned in row i and column j in matrix A stores the distance between node i and node j in graph G. Draw the graph and solve the single-source-shortest path problem by running the Dijkstra's algorithm on G, starting at node 1. What is the order in which nodes get removed from the associated priority queue? What is the resulting shortest-path tree?

- **Part E**

Let P be a shortest path from some vertex s to some other vertex t in a graph. If the weight of each edge in the graph is increased by one, then will P be still a shortest path from s to t? Explain your answer.

- **Part F**

You are given a graph G with exactly one cycle of negative weight (cost). You know the vertices that constitute the cycle. Design an algorithm that checks whether there exists a path from the source node to the destination with a negative total weight. You must explain the runtime complexity of your algorithm.

Further Notes

- The lecture recordings for weeks 8 and 9 should serve as a good starting point to understand the relevant graph traversal algorithms and the Shortest Path Problem.
- The following chapters of the course book "Data Structures and Algorithms in Java" by Michael T. Goodrich, Irvine Roberto Tamassia, and Michael H. Goldwasser (2014) should help you with the answers to this task.
  - Parts A-C: Chapters 14.1-14.3
  - Parts D-E: Chapter 14.6

  You may access the book on-line for free from the reading list application in CloudDeakin available in Content → Reading List → Course Book: Data structures and algorithms in Java.

Submission Instructions and Marking Process

To get your task completed, you must finish the following steps strictly on time.

- **Submit** your answers as a single PDF report via OnTrack submission system and **record a video explaining your solution** to the task. Remember to explain the designs and pseudocodes of your algorithms. Upload the video to one of accessible resources and refer to it for the purpose of marking. You must provide a private working link to the video to your marking tutor in OnTrack. Note that the video recording must be made in the **camera on mode**; that is, the tutor must see both the presenter and the shared screen.
- Once your solution is accepted by the tutor, you will be invited to **continue its discussion and answer relevant theoretical questions** through the Intelligent Discussion Service in OnTrack. Your tutor will record several audio

questions. When you click on these, OnTrack will record your response live. You must answer straight away in your own words. As this is a live response, you should ensure you understand the solution to the task you submitted. Answer all additional questions that your tutor may ask you. Questions will cover the lecture notes; so, attending (or watching) the lectures should help you with this **compulsory discussion part**. You should start the discussion as soon as possible as if your answers are wrong, you may have to pass another round, still before the deadline. Use available attempts properly.

Note that we will not accept your solution after **the submission deadline** and will not discuss it after **the discussion deadline**. If you fail one of the deadlines, you fail the task, and this reduces the chance to pass the unit. Unless extended for all students, the deadlines are strict to guarantee smooth and on-time work throughout the unit.

Remember that this is your responsibility to keep track of your progress in the unit that includes checking which tasks have been marked as completed in the OnTrack system by your marking tutor, and which are still to be finalised. When grading your achievements at the end of the unit, we will solely rely on the records of the OnTrack system and feedback provided by your tutor about your overall progress and the quality of your solutions.